مستندات کامل پیاده‌سازی

انتخاب ویژگی هزینه‌آگاه و پایدار مبتنی بر EHOA

پروژه درس

کلان‌داده - کارشناسی ارشد

امیررضا غفاریان — دانشجو:
دکتر محمدحسین معطر — استاد:
دانشگاه آزاد اسلامی — دانشگاه:

مرداد ۱۴۰۵ — تاریخ تهیه:

نسخه نهایی قابل بازتولید

فهرست

1 هدف و دامنه 2

2 ساختار نرم‌افزار 2

3 منطق روش پیشنهادی 2
3.1 تعمیر دقیق قید بودجه 2
3.2 برآورد سودمندی پایدار 3
3.3 ادغام با EHOA 3

4 پروتکل اجرا و جلوگیری از نشت داده 3

5 آزمون‌ها و کنترل کیفیت 4

6 روش بازتولید 4

7 محدودیت‌های فنی 4

8 جمع‌بندی 4

1 هدف و دامنه

این پروژه یک انتخاب‌گر ویژگی پوششی بر پایه الگوریتم بهینه‌سازی کوه‌پیمایی بهبودیافته (EHOA) را پیاده‌سازی می‌کند. توسعه پیشنهادی با نام CostStableEHOA یک قید سخت بودجه را به فرایند جست‌وجو اضافه می‌کند و هنگام نقض بودجه، ویژگی‌ها را بر اساس نسبت «اطمینان محافظه‌کارانه از سودمندی» به هزینه حذف می‌کند. تابع برازندگی پایه تغییر نکرده است؛ در نتیجه اثر سازوکار تعمیر را می‌توان مستقل از تغییر هدف ارزیابی کرد.

ادعای فنی محدود است: ترکیب تعمیر بودجه‌ای با امتیاز LCB حاصل از بازنمونه‌گیری اطلاعات متقابل، درون EHOA. هزینه‌آگاهی، اطلاعات متقابل و تعمیر جواب به‌تنهایی نوآوری محسوب نمی‌شوند.

2 ساختار نرم‌افزار

فایل	مسئولیت
proposed/ehoa.py	هسته EHOA، مقداردهی جمعیت، تبدیل دودویی، ارزیابی و به‌روزرسانی عامل‌ها.
proposed/cost_stable_ehoa.py	محاسبه سودمندی پایدار، تعمیر قید بودجه و کلاس روش پیشنهادی.
run_cost_stable_confirmatory.py	اجرای ارزیابی تودرتو، ذخیره مرحله‌ای ردیف‌ها و ماسک‌ها و قابلیت ادامه اجرا.
analyze_cost_stable_confirmatory.py	کنترل صحت، تجمیع، آزمون‌های فریدمن/ویلکاکسون/هولم و رسم نمودارها.
configs/cost_stable_confirmatory.yaml	تمام پارامترها، بذرها، دیتاست‌ها، بودجه و روش‌ها.
tests/test_proposed.py	آزمون واحد تعمیر، رعایت بودجه، قطعیت و رفتار الگوریتم.

3 منطق روش پیشنهادی

برای ویژگی j و بازنمونه‌گیری b، اطلاعات متقابل I_{jb} محاسبه می‌شود. امتیاز محافظه‌کارانه برابر است با

$$u_j = \max\left(0, \bar{I}_j - z\, s_j\right), \qquad r_j = \frac{u_j}{c_j}. \tag{1}$$

اگر هزینه ماسک $\sum_j z_j c_j$ از بودجه B بیشتر باشد، ویژگی فعال با کوچک‌ترین r_j حذف می‌شود تا قید برقرار شود. استفاده از کران پایین باعث می‌شود ویژگی‌ای که میانگین سودمندی بالا ولی نوسان زیاد دارد، بی‌دلیل حفظ نشود.

3.1 تعمیر دقیق قید بودجه

```
def repair_to_budget(mask, costs, budget, utility):
    repaired = np.asarray(mask, dtype=bool).copy()
    while costs[repaired].sum() > budget:
        active = np.flatnonzero(repaired)
```

```
        ratio = utility[active] / np.maximum(costs[active], 1e-12)
        repaired[active[np.argmin(ratio)]] = False
    if not repaired.any():
        feasible = np.flatnonzero(costs <= budget)
        repaired[feasible[np.argmax(utility[feasible] / costs[feasible
])]] = True
    return repaired
```

این قطعه ابتدا یک کپی بولی می‌سازد تا ورودی تغییر نکند. حلقه فقط تا زمان نقض بودجه ادامه دارد. در هر دور، ضعیف‌ترین ویژگی از نظر سودمندی به‌ازای هزینه حذف می‌شود. شرط پایانی از ماسک تهی جلوگیری می‌کند و ارزان‌ترین گزینه را کورکورانه انتخاب نمی‌کند؛ بهترین نسبت سودمندی به هزینه میان گزینه‌های شدنی انتخاب می‌شود.

3.2 برآورد سودمندی پایدار

```
scores = []
for _ in range(n_resamples):
    idx = rng.choice(len(y), len(y), replace=True)
    scores.append(mutual_info_classif(X[idx], y[idx], random_state=seed
))
scores = np.asarray(scores)
utility = np.maximum(0.0, scores.mean(0) - lcb_z * scores.std(0, ddof
=1))
```

بازنمونه‌گیری فقط روی بخش آموزش بیرونی انجام می‌شود؛ داده آزمون بیرونی در محاسبه امتیاز یا جست‌وجو حضور ندارد. میانگین، قدرت تقریبی رابطه و انحراف معیار، عدم‌قطعیت آن را نشان می‌دهد. پارامتر z شدت محافظه‌کاری را کنترل می‌کند.

3.3 ادغام با EHOA

```
class CostStableEHOA(EHOA):
    def _repair(self, mask):
        return repair_to_budget(mask, self.costs, self.budget, self.
utility_)

    def fit(self, X, y):
        self.utility_ = resampled_lcb_utility(X, y, random_state=self.
random_state)
        return super().fit(X, y)
```

وراثت، تکرار کد پایه را کم می‌کند. تنها نقطه مداخله، تعمیر هر جواب کاندید پیش از ارزیابی است. بنابراین اندازه جمعیت، قوانین حرکت، تبدیل دودویی و تابع برازندگی همان EHOA باقی می‌مانند و مقایسه کنترل‌شده است.

4 پروتکل اجرا و جلوگیری از نشت داده

ده دیتاست، پنج روش، دو بذر مستقل و سه چین بیرونی به $10 \times 5 \times 2 \times 3 = 300$ اجرا منجر شده است. پیش‌پردازش و طبقه‌بند فقط با داده آموزش هر چین برازش می‌شوند. بودجه ۳۰ درصد مجموع هزینه‌هاست؛ هزینه‌ها به‌صورت قطعی از توزیع لگ‌نرمال و مستقل از برچسب تولید شده‌اند. هر ماسک، بذر انتخاب‌گر، هش پیکربندی، زمان و معیارهای آزمون ذخیره شده‌اند.

5 آزمون‌ها و کنترل کیفیت

مجموعه آزمون شامل رعایت قید، عدم تولید ماسک تهی، قطعیت در بذر ثابت، صحت ابعاد و سازگاری با EHOA است. نتیجه نهایی: ۲۸ آزمون از ۲۸ آزمون موفق. کنترل پس از اجرا نیز ۳۰۰ ردیف، ۳۰۰ ماسک، صفر کلید تکراری، صفر مقدار گم‌شده و صفر نقض بودجه را تأیید کرد.

6 روش بازتولید

```
python -m pip install -r requirements.txt
python -m pytest -q
python run_cost_stable_confirmatory.py \
  --config configs/cost_stable_confirmatory.yaml \
  --output results/cost_stable_confirmatory
python analyze_cost_stable_confirmatory.py
```

فایل اجرا پس از هر آزمایش CSV را به‌روز می‌کند؛ در صورت قطع برق یا توقف برنامه، گزینه resume فقط کلیدهای کامل‌شده را رد می‌کند. این ویژگی مانع از دست‌رفتن محاسبات طولانی می‌شود.

7 محدودیت‌های فنی

هزینه‌ها واقعیِ دامنه پزشکی یا صنعتی نیستند و برای کنترل آزمایش به‌صورت مصنوعی اما ثابت ساخته شده‌اند. تنها طبقه‌بند گزارش اصلی KNN است. تعداد بذرها دو و تعداد چین‌های بیرونی سه است؛ بنابراین واحد آزمون بین‌دیتاستی ده دیتاست است و نباید ۳۰۰ ردیف را به‌اشتباه ۳۰۰ مشاهده مستقل تلقی کرد.

8 جمع‌بندی

پیاده‌سازی نهایی آزمون‌پذیر، قابل ادامه و قابل ممیزی است. نوآوری در سطح سازوکار ترکیبی و افزایشی مطرح می‌شود و کد اجازه می‌دهد چهار تعمیر تصادفی، ارزان‌ترین، اطلاعات متقابل و Stable-LCB تحت بودجه یکسان مقایسه شوند.